

NIDS PROJECT

Beginner's Complete Guide

Everything you need to understand before writing code

Telecom / Infrastructure Engineer Track

7 Concepts

Step-by-step

NIDS Focused

Table of Contents

- | | |
|---|--|
| 1 | HTTP & REST APIs — How the Internet Talks |
| 2 | Docker & Docker Compose — Your Project's Engine Room |
| 3 | PostgreSQL — Storing Alerts Forever |
| 4 | Redis — The Speed Layer |
| 5 | WebSockets — Real-Time Alerts |
| 6 | FastAPI — Building Your Backend |
| 7 | Scapy — Capturing Network Packets |
| 8 | How Everything Connects in NIDS |
| 9 | Week 1 Checklist |
-

1 — HTTP & REST APIs

What problem does it solve?

When two programs on different machines need to talk to each other, they need a shared language. HTTP (HyperText Transfer Protocol) is that language. REST (Representational State Transfer) is a set of rules for how to use HTTP in a clean, predictable way.

The Restaurant Analogy

■ Think of your NIDS system as a restaurant. The React dashboard is the customer. FastAPI is the waiter. The ML model is the chef. HTTP is the language the customer uses to order food.

HTTP Methods you will use

Method	What it does in NIDS
GET	Dashboard asks: 'Give me the last 50 alerts'
POST	Dashboard sends 78 features, asks: 'Is this an attack?'
DELETE	Admin deletes old alerts from the database

What is JSON?

JSON (JavaScript Object Notation) is the format used to send data between your frontend and backend. It looks like a Python dictionary. When the dashboard sends 78 network features to FastAPI, it sends them as JSON:

```
{
  "features": [0.5, 1.2, 0.0, 3.7, ... ]  // 78 numbers
}
```

FastAPI replies with JSON:

```
{
  "prediction": "DDoS",
  "confidence": 0.97,
  "attack_type": "DoS/DDoS"
}
```

What is an Endpoint?

An endpoint is simply a URL address on your server that does a specific job. In your NIDS project, you will build these endpoints:

Endpoint	Job
POST /api/predict	Receive 78 features, return attack prediction
GET /api/alerts	Return list of past alerts from database
GET /api/stats	Return traffic statistics for the dashboard
WS /ws/alerts	WebSocket — push live alerts to dashboard

2 — Docker & Docker Compose

What problem does it solve?

'It works on my machine' is the most famous problem in software. Docker solves it by packaging your application and all its dependencies into a container — a lightweight, isolated box that runs identically everywhere: your Kali Linux, your friend's Ubuntu, a cloud server.

Container vs Virtual Machine

Container (Docker)	Virtual Machine
Shares the host OS kernel	Has its own full OS
Starts in milliseconds	Starts in minutes
Uses ~50MB RAM	Uses ~1-2GB RAM
Perfect for microservices	Better for full OS isolation

What is a Dockerfile?

A Dockerfile is a recipe that tells Docker how to build your application's image. Your backend Dockerfile:

```
FROM python:3.11-slim      # Start from Python image

WORKDIR /app               # Work inside /app folder

COPY requirements.txt .    # Copy your dependency list
RUN pip install -r requirements.txt # Install dependencies

COPY . .                  # Copy all your code

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

What is Docker Compose?

Your NIDS project has 3 services that need to talk to each other: the FastAPI backend, PostgreSQL database, and Redis cache. Docker Compose lets you define and start all 3 with a single command.

`docker compose up --build` This one command: builds your backend image, downloads PostgreSQL and Redis images, starts all 3 containers, connects them on a private network, and maps ports to your machine.

The 3 services in your docker-compose.yml

Service	Role in NIDS
backend	Your FastAPI app — handles predictions and alerts
db	PostgreSQL — stores all alerts permanently on disk
redis	Redis — temporary fast storage for real-time data

3 — PostgreSQL

What problem does it solve?

When your NIDS detects an attack, you need to save that information permanently — even after the server restarts. PostgreSQL is a relational database that stores data in tables (like Excel spreadsheets) and lets you query it with SQL.

Your alerts table will look like this:

Column	Type Example value
id	Integer 1
timestamp	DateTime 2026-06-07 01:10:08
src_ip	String 192.168.1.45
dst_ip	String 10.0.0.1
attack_type	String DDoS
confidence	Float 0.97
severity	String HIGH

Essential SQL you need to know

```
-- Get the last 10 alerts
SELECT * FROM alerts ORDER BY timestamp DESC LIMIT 10;

-- Count attacks by type
SELECT attack_type, COUNT(*) FROM alerts GROUP BY attack_type;

-- Get only HIGH severity alerts
SELECT * FROM alerts WHERE severity = 'HIGH';
```

Why not just use a file (CSV)?

A CSV file would work for 100 alerts. But your NIDS will detect thousands of events per hour. A database handles concurrent writes, lets you query by any column instantly, and never corrupts data even if the server crashes mid-write.

4 — Redis

What problem does it solve?

PostgreSQL stores data on disk — reading it takes milliseconds. Redis stores data in RAM — reading it takes microseconds (1000x faster). For real-time features like live alert counts and recent traffic stats, speed matters.

PostgreSQL vs Redis — clear comparison

PostgreSQL	Redis
Stores on disk (permanent)	Stores in RAM (temporary by default)
Milliseconds to read	Microseconds to read
Complex queries (SQL)	Simple key-value lookups
Use for: all alerts history	Use for: last 10 alerts, live counters

How Redis works in your NIDS

Every time an attack is detected, your backend does two things simultaneously:

```
# 1. Save to PostgreSQL (permanent record)
db.add(alert)
db.commit()

# 2. Push to Redis (real-time notification)
redis_client.publish("alerts", alert_json)
# Dashboard receives this instantly via WebSocket
```


5 — WebSockets

What problem does it solve?

With regular HTTP, your React dashboard would have to ask the server every second 'any new alerts?' — this is called polling. It's wasteful. WebSocket creates a permanent two-way connection: the server pushes alerts the instant they happen.

HTTP polling vs WebSocket

HTTP Polling	WebSocket
Dashboard asks every second	Server pushes when alert happens
99% of requests = no new data	Every message = real data
Delay up to 1 second	Delay under 50ms
Heavy on server resources	One persistent connection

The flow in your NIDS

```
Packet captured by Scapy
|
78 features extracted
|
FastAPI sends to ML model
|
Model says: DDoS (confidence 0.97)
|
FastAPI saves alert to PostgreSQL
|
FastAPI pushes alert via WebSocket
|
React dashboard shows alert INSTANTLY
--> Red notification appears on screen
```

6 — FastAPI

What problem does it solve?

FastAPI is a Python framework for building REST APIs. It handles HTTP requests, validates input data automatically, generates interactive documentation, and is one of the fastest Python frameworks available — critical when processing network packets in real time.

Why FastAPI and not Flask or Django?

Feature	FastAPI Flask Django
Speed	Very fast Moderate Moderate
Auto docs (Swagger)	Yes No No
Data validation	Built-in Manual Manual
WebSocket support	Native Plugin Plugin
Best for	APIs Simple Full web

Your main.py explained line by line

```
from fastapi import FastAPI          # Import the framework
from app.routes.predict import router # Import your prediction route

app = FastAPI(title='NIDS API')       # Create the application

app.include_router(router, prefix='/api')# Register the /api/predict endpoint

@app.get('/')                        # Define the / endpoint
def health_check():                  # Function that runs when / is called
    return {"status": "NIDS API running"}# Return JSON response
```

Swagger UI: When FastAPI runs, it automatically generates interactive documentation at <http://localhost:8000/docs> — you can test all your endpoints directly from the browser without writing any frontend code. This is extremely useful during development.

7 — Scapy

What problem does it solve?

Your ML model was trained on CICIDS2017 — a dataset of network features extracted from real packets. To detect attacks on live traffic, you need to capture actual packets and extract those same 78 features. Scapy is a Python library that lets you do exactly that.

What Scapy captures from each packet

Feature category	Examples
Flow duration	How long the connection lasted
Packet lengths	Min, max, mean, std of packet sizes
Inter-arrival time	Time between packets (IAT)
Flag counts	Number of SYN, ACK, FIN, RST, PSH flags
Byte rates	Bytes per second in each direction
Port info	Source port, destination port, protocol

The pipeline from packet to prediction

```
# 1. Scapy captures a raw packet
packet = sniff(count=1)[0]

# 2. Extract features from the packet
features = extract_78_features(packet) # Your function

# 3. Send to FastAPI for prediction
response = requests.post("http://localhost:8000/api/predict",
                        json={"features": features})

# 4. If attack detected – create alert
if response.json()["prediction"] != "BENIGN":
    create_alert(response.json())
```

8 — How Everything Connects in NIDS

The complete data flow

```
NETWORK TRAFFIC
  |
  v
[Scapy] captures raw packets
  |
  v
[Feature Extractor] produces 78 numbers per flow
  |
  v
[FastAPI /api/predict] receives the 78 features
  |
  v
[ML Model] classifies: BENIGN / DDoS / PortScan / ...
  |
  +----> [PostgreSQL] stores alert permanently
  |
  +----> [Redis] publishes to alert channel
          |
          v
        [WebSocket] pushes to connected clients
          |
          v
    [React Dashboard] shows real-time alert
```

Your responsibilities vs your teammate's

You (Telecom Engineer)	Teammate (ML Engineer)
FastAPI backend architecture	CICIDS2017 data exploration
Docker Compose + all containers	Random Forest + XGBoost models
PostgreSQL schema + queries	Hyperparameter tuning
Redis real-time layer	Model evaluation report
WebSocket alert push	Feature importance analysis
React dashboard + charts	Neural network comparison
Scapy packet capture module	Model export with joblib
End-to-end pipeline assembly	Prediction endpoint logic

9 — Week 1 Checklist

Days 1–3 checklist

- Git + Python + Docker installed and verified
- GitHub repo nids-ai created
- Project folder structure created
- requirements.txt filled with 7 dependencies
- backend/app/main.py written
- backend/app/routes/predict.py written
- backend/Dockerfile written
- docker-compose.yml written with 3 services
- docker compose up --build runs without error
- localhost:8000 returns health check JSON
- localhost:8000/docs shows Swagger UI

Days 4–5 checklist

- Endpoint accepts list of 78 floats (Pydantic model)
- Returns prediction + confidence + attack_type
- Returns error if not exactly 78 features
- Tested via Swagger UI
- All code pushed to GitHub

Remember the 20-minute rule from your project plan: when you hit something you don't understand, set a 20-minute timer, understand the concept, write 3 sentences about it, then keep building. Don't let one unknown block a whole day.